

Controllable Procedural Generation of Race Track Surroundings for Iterative Level Design

1st Petr Šimůnek

*Faculty of Mathematics and Physics
Charles University
Prague, Czech Republic
petr.simunek@matfyz.cuni.cz*

2nd David Šosvald

*Faculty of Mathematics and Physics
Charles University
Prague, Czech Republic
sosvald@ksvi.mff.cuni.cz*

3rd Martin Kahoun

*Faculty of Mathematics and Physics
Charles University
Prague, Czech Republic
kahoun@cgg.mff.cuni.cz*

4th Vojtěch Černý

*Faculty of Mathematics and Physics
Charles University
Prague, Czech Republic
cerny@gamedev.cuni.cz*

Abstract—Modern racing games often take place on race tracks with huge surroundings, including forests, fields, cities, or mountains. While not being important to the racing itself, the surroundings must be present, dense, and detailed to enhance the gameplay feel, and must be performance-optimized for a smooth gaming experience. Creating these surroundings manually is a very laborious task, even for a skilled level designer. Also, manual design makes it prohibitively expensive to modify the track layout later. This makes the surroundings an ideal candidate for procedural content generation, which is nowadays extensively used in many areas of video game level design. In this paper, we present a method for controllable track surroundings generation. It generates a one square kilometer map in a matter of minutes, orders of magnitude faster than the manual process. It features several different biomes, various types of assets and vegetation, and can be further extended using custom assets. The generated race tracks and their surroundings are easily adjustable, making the level design iterative and easier to perfect.

Index Terms—procedural generation, level design, racing games, runtime optimization, controllable generation

I. INTRODUCTION

In a typical modern racing game, race track surroundings are a significant and necessary feature, requiring substantial development effort. Although unimportant to the game mechanics, since the main gameplay happens on the race track, there are multiple requirements for the track surroundings to fulfill. We designed our work with both the players playing the game and the level designers creating the game content in mind.

For the players, the surroundings must be good-looking and performant. A big part of the racing game is the atmosphere; players want to feel connected to the world and have a sense of speed thanks to the objects around the track. The perceived quality and enjoyment are highly subjective to each player.

The work was supported by the grant SVV–2025–260822.

Hence, a good race track requires much fine-tuning by the designer.

For the designers, we can split the process of creating a race track into two main parts: creating the layout of the track, and creating the track surroundings. First, the designer draws the track's layout based on their subjective goals; this process is iterative. Sometimes, the designer can change the track design even after public release based on player feedback. The track design usually takes between several minutes and a few hours of a designer's time.

The next step is creating the track surroundings, which can take from several dozen to hundreds of hours. However, it has minimal impact on the gameplay itself. Also, if the track's layout is changed, the surroundings need to be reworked entirely. Hence, it makes a lot of sense to automate this process. A simple showcase of how our algorithm works is shown in Figure 1.

The main gameplay dramatically depends on the track layout. Therefore, we firmly believe that the track layout needs to be precisely controlled by the designer. Also, the track layout can be designed in a relatively short time; the surroundings are the time bottleneck, and they need to be well-optimized as well. Our approach offers a novel perspective by placing greater emphasis on the surroundings rather than the race track itself, a focus that appears to be less prevalent in existing works [1]–[5].

A. Goals

Our work aims to present an algorithm that significantly speeds up the workflow of creating race tracks and their surroundings while enabling designers to maintain complete control over the gameplay. We have implemented the algorithm using the Unity game engine [6], but the approach is usable in any similar tech stack. The individual goals that we set are the following:



Fig. 1. Comparison between the input race track (top) and the resulting race track with generated surroundings (bottom).

1) *Controllable generation*: The goal of the generator is to allow the level designer to create and iterate a race track with full control of its shape and width, while automatically generating the surroundings of the track. The generator should provide control over the biomes and assets spawned.

2) *Generator performance*: The system needs to be performant enough to allow for quick iterations of the surroundings. This means that the whole generation process should not take more than a few minutes.

3) *Runtime performance*: The map generated by the system needs to be runtime-optimized, allowing it to be used out-of-the-box in a real time racing game. Special attention will be given to optimizing the density of assets in regard to their distance from the track.

II. RELATED WORK

Much research has been done regarding race track procedural generation, resulting in numerous publications focusing on modeling specific features or structures. However, few have focused on complex systems, which necessitate the integration of multiple methods for different elements, such as whole race track surroundings.

A. Terrain generation

In recent years, many methods for generating realistic terrains for video games have emerged [7]–[9], utilizing various simulation or modeling-based methods. Although these works achieve satisfactory results, they lack the option of generation denser environment around points of interest. Further, many of these tools lack the runtime performance required for real time video games.

B. Race track modeling

One of the commercially available projects most similar to our work is the *Race Track Generator* [1]. The difference is that this project focuses on generating race tracks used by the top levels of motorsport, and that it does not generate further track surroundings. We focus on *road racing* tracks located in everyday locations like villages, forests, and fields; the *Race Track Generator* does not model these biomes. This solution also does not support varying width of the track. Further, we target a denser environment.

There is also a project of the same name available on GitHub, the *Race Track Generator* [2]. It uses Voronoi cells [10], and the race track is based on the edges of the cells. The results look pleasant and their shape can be easily modified to suit designers' preferences. However, it does not provide precise control over the width of the track, and the tool does not generate the track surroundings, landscape, or infrastructure, nor is there an option for drawing objects other than the track.

A similar tool to the previous one is *Procedural Racetrack Generation* [3]. This tool produces visually pleasing layouts of race tracks. But similarly to the other works, the track width cannot be edited, and the tool does not generate the track surroundings or infrastructure.

Research has also been done into generating race tracks by utilizing evolutionary methods [4], [11] to generate the shape of the track and its simple surroundings. Compared to our goals, the surroundings generated by these works are too sparse, the visual fidelity is not high enough, and the designer has no control over the track's shape.

C. Nature generation

One of the most prominent tools for nature generations is the *Vegetation Studio Pro* [12], a package for high-performance placement and rendering of vegetation, available on the Unity asset store. After our analysis, we concluded that it has many great features, but also several problems. It is great in terms of rule-based generation, and it is possible to define so-called vegetation masks to exclude vegetation from certain places. It is also possible to create different biomes to make the parts of the track feel special.

On the other hand, different density levels could be done only by introducing new biomes. If two biomes used the same prefab, they could not be drawn at once. Due to compatibility issues and dependencies, it is also quite cumbersome to use. Compatibility with custom shaders is complicated, and although billboards are generated, there is no option to have custom ones. We have opted to implement our own vegetation module for this work.

There exist other tools for realistic nature generation, e.g., the forest generator by Newlands and Zauner [13], but these lack the performance required for real time use.

D. Procedural content generation in general

A *Survey on the Procedural Generation of Virtual Worlds* [14] contains many relevant subjects such as terrain texturing

workflow [15], [16], spline-based editor [17], and Voronoi diagrams [10]. This article also discusses the performance tradeoffs which come in helpful when optimizing performance or preparing prefabs for a demo game. The article suggests controlling the density of the vegetation with a grayscale map. For tree placing, it suggests to use *Poisson disk distribution* [18] because of its simplicity and speed.

Track generation is discussed as well. The article proposes using *Bézier curves* [19] and put geometry at regular intervals based on interpolation parameters. In our opinion, this produces wasteful amounts of geometry in some places and insufficient in others (e.g., sharp turns or corners). Our work solves this issue by dynamically changing the sampling rate (details in Section IV-A).

III. OVERVIEW

Let us now describe how our generator can be controlled and then provide an overview of the generation, dividing it into individual steps.

A. Generator control workflow

The workflow starts with the designer creating an outline of the track. We have implemented a Unity Editor tool that allows the designer to model the individual segments of the track separately. An example of the user interface for track modeling can be seen in Figure 2. Each of the segments has a control point that determines the segment's position, tangent, and the width of the track at that point. The width of the track in the remainder of the track is interpolated between the control points. Curves of general shapes and crossroads of any shape with changing road surfaces can be modeled. The editor takes care of connecting the track segments into the final circuit. At this point, the ideal line through the track can also be specified to be further used for AI controlled opponents and for user interface.

After the outline of the track is finished, the generator is tasked with creating the surroundings of the track, including cities, forests, meadows, groups of fans, vegetation, and much more. When the generation is complete, the designer can arbitrarily change the track's shape. They can also overwrite the biomes generated with different ones, and it is possible to change the types of assets generated (types of houses, sidewalks, fences, etc.). The generator is then tasked with overwriting the track surroundings based on the new settings. These iterations between the designer and the generator continue until the designer is satisfied with the result.

B. High-level overview

The algorithm consists of six distinct generation steps depicted in Figure 3. Each step is associated with a specific data type or algorithm, and its respective output. These steps are discussed in more detail in further sections.

C. Circuit layout

Before we begin any procedural generation, we need the designer to draw the race track. We opted for a system based

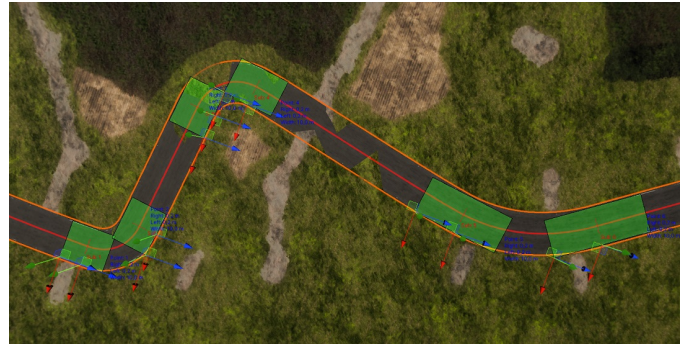


Fig. 2. An example of the track editor's user interface. We can see the individual track segments with their control points. The tool automatically interpolates the track's curvature between them.

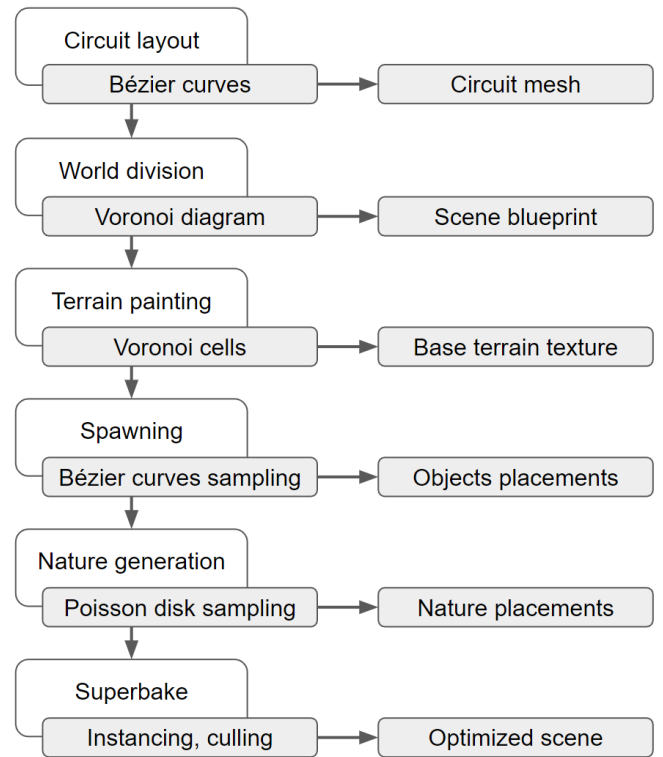


Fig. 3. The main steps of our approach, including the type of data that each step produces.

on *Bézier curves* [19]. Other curves are also possible to use, for example, B-spline [20]. We chose Bézier curves since they are intuitive for users and relatively familiar in the industry; they also represent real-world shapes common for race tracks well. In our implementation, each control point has an associated track width, which gives the user great control over the final result.

D. World division

In the next step, we proceeded to divide the world into cells, which are the base for different biomes like cities, forests, and fields. Since our tracks have irregular shapes, we opted to use

Voronoi diagrams for this task. There are other alternatives; a square grid is suitable for race tracks located in cities, as are a triangle grid, a hexagonal grid, etc.

First, we place the centers of the cells on the edges of the track. This splits the track into two separate parts, giving us the ability to have different biomes on each side of the track. Then, we assign different biomes to each cell based on *Perlin noise* [16]. We also decide where secondary roads will be placed; we use the Voronoi graph edges for them. Finally, this step determines the position of houses, fans, curbs, safety nets, etc.

E. Terrain painting

We write the resulting data into the scene by painting the base terrain. Each biome cell is painted onto the terrain using its associated splat texture.

F. Spawning

The next step is transferring the data generated in step two into the scene. In this step, we draw the secondary roads, detect roads in the cities and add sidewalks, etc. Houses are generated in cities and bushes are spawned along the roads in the countryside. Lastly, additional terrain painting takes place to fit newly spawned features in.

G. Nature generation

When the terrain texture is done, we use it to place trees, bushes, and grass. We implement vegetation rules associated with the terrain texture, dictating vegetation density controlled by the distance from the race track. We then detect the edges of the biomes and make all vegetation denser there in order to increase visual fidelity. We use the distance from the track since the player spends most of the time on the race track or close to it. We utilize high-precision vegetation masks to remove vegetation from unwanted areas, such as roads and sidewalks. Texture arrays are utilized to support many different vegetation types while reducing the number of draw calls.

H. Superbake – pre-runtime optimizations

In the last step, we exploit the repetitive nature of the track surroundings. Reused meshes are serialized for GPU instancing and removed from the scene to save disk space. This reduces draw calls, boosting performance and load times. Unreachable colliders are culled to improve CPU performance. Lastly, we generate metadata, including a global minimap, and bake occlusion culling.

IV. IMPLEMENTATION

We have designed a simple track with distinct sections, non-monotone turns, and overtake opportunities to demonstrate our algorithm in the following section. The track shape can be seen in Figure 4. The following sections describe the steps presented in Section III in more detail.

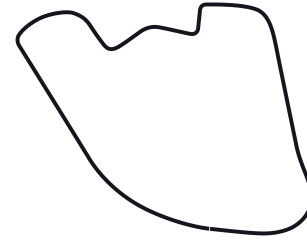


Fig. 4. An example layout of a track. This layout will be used in the subsequent examples.

A. High-performance Bézier curve sampling

To render the surface of the race track smoothly, we need to precisely sample the Bézier curve that represents the shape of the race track. Such sampling also needs to be performant to enable a quick iteration cycle. We use two different approaches for curve sampling: regular and adaptive. The regular approach samples points on the curve in regular intervals, and is used for placing assets (e.g., guardrails and poles).

The adaptive approach is used for geometry generation (e.g., roads and sidewalks), where the curvature of the surface needs to be considered, i.e., the samples can be further apart when the track is straight, and more samples are needed to accurately model the turns of the track. This approach takes the derivative of the curve (the angle between the tangents of the two points) into consideration.

Regular sampling enforces a fixed distance between consecutive samples, while the adaptive approach adds an additional condition based on the change in angle between samples. This ensures denser sampling in regions of higher curvature. To estimate the Bézier curve's length, we select a few representative points and calculate the distances between them. Using this rough length estimate, we determine an initial step size for sampling the curve. As we sample, we check whether the distance and angle conditions are satisfied. When a condition is about to be violated (such as nearing a sharp turn) we reduce the step size to increase the sampling density. Finally, we cache previously computed samples to avoid redundant calculations.

B. Distance function initialization

As many subsequent generation steps need a performant function to calculate the distance of points from the track, the algorithm starts by creating a 2D grid of the same size as the terrain, located in the terrain's local space. We will precompute the distance of the grid's vertices to the track.

We start by sampling points on the main Bézier curve because this information is needed for the computation of the distance function. We store the world space position, road tangent, normal, and width at each sampled point. Now we can compute the distances of the grid vertices. The size of the cell was selected to be 1 meter $\times$ 1 meter, but other sizes would

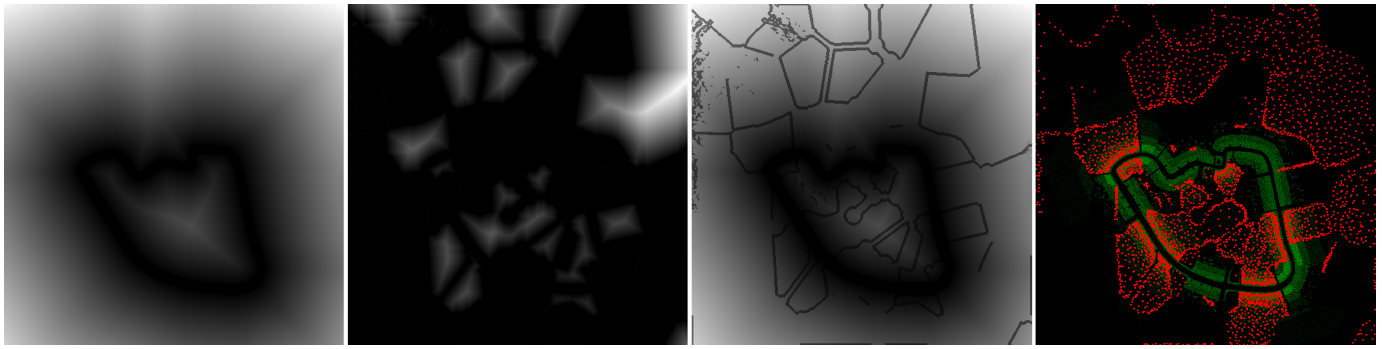


Fig. 5. The distance functions visualized are from left to right: 1) distance from the track, 2) distance from the nearest non-forest area (used to make forests denser around the edges), 4) density of trees (red) and grass (green). The black areas have a distance of zero from the object of interest, and the white areas are far from it. The third function is computed by dividing the distance function values at the edges of the forests by a constant. The fourth function shows the density of the generated nature; it is visible that the density decreases with increasing distance from the track, with tree density increasing on the edges of forests.

work too. We start by utilizing the samples of the track and assigning the value of zero to all the points inside the track.

The remaining points are assigned the value of infinity. We then propagate the distances from the track using breadth first search and finish by constructing a function to access the grid from world space. We achieve this by transforming the world space points into terrain space points and finding the nearest points of the grid, which is done by simple rounding. We can see a visualization of the distance function in Figure 5.

C. Voronoi cell generation

Now comes the critical part of the algorithm; we generate the Voronoi diagram that divides the world into cells. It is worth noting that the world generation method is independent of the cell creation scheme. It could be a square grid, triangle grid, etc. We chose the Voronoi diagram because it can produce variable shapes and each cell can be different, so that the shape of the track can be quite complex. The results produced by the Voronoi diagram look the most natural for our use case.

We start by generating centroids (the center points of the Voronoi cells). Two types of centroids are used, the first type follows the road to ensure that biomes change along the track, and the second type is randomly generated in the remaining parts of the terrain.

The first type follows the race track so that the Voronoi cell edges follow it too and split the race track in the middle to enable the generation of different biomes on the opposite sides of the race track. More centroids are needed around the turns of the track to ensure that their edges still split the track around the middle. We use adaptive sampling (described in Section IV-A) to achieve this.

In the second stage, the remaining centroids are generated randomly, but they must be further away from the track than what is the minimal distance parameter from Bézier curve sampling divided by two. This ensures that we do not interfere with the Voronoi cells surrounding the track in an undesired manner. The example of the resulting centroids can be seen in Figure 6.



Fig. 6. Voronoi centroids visualization, each dot is a centroid. The color scheme is the following: red = city, white = meadow, yellow = field, green = forest, black = clearing, grey = small dense forest.

D. Spawning

With the Voronoi diagram initialized, we generate side streets and mud roads along the edges of neighboring Voronoi cells. We then sample both the main curves and the edges of all the roads regularly. Each sample is represented by its world space position, tangent, and normal. The algorithm now assigns assets to the edge samples based on hand-crafted rules. Several examples of the side objects include curbs, highlighted curbs, safety nets, fans, tire walls, and houses. The asset spawning rules take into account the road curvature, the distance from the nearest corner, the presence or absence of other objects, the current biome, and much more.

E. Nature generation

The next stage involves vegetation generation around the track, categorized into two types: trees and grass. We first address tree placement, then extend and optimize the method for grass, which poses greater challenges due to a higher instance density.

The idea of the tree generating algorithm is as follows: we aim to spawn denser vegetation close to the track in order to



Fig. 7. Mask precision showcase. As we can see, it is possible to have dense vegetation close to the road's edge. We are limited only by the float precision and the quality of prefabs.

effectively utilize its instances (the players spend most of their time on the track). This is achieved by changing the density of the spawned vegetation depending on the distance from the track – denser close to the track and sparser far from the track.

We utilize cascades to achieve this – each cascade is defined by the minimum and maximum distance from the track and has an associated density (a concept similar to shadow cascades). Visualization of the cascades is shown in Figure 5. The cascades are connected to each other, starting from the track (distance of zero) up to the maximal distance (end of terrain). The next step is to generate points (where the vegetation will be spawned) inside the cascades with the density defined by each cascade. We use Poisson disk sampling [18] because it's a sufficient approximation of vegetation density in the real world, though other algorithms could yield similar results [21], [22].

The generation of points for each cascade is done by computing a bounding box of the cascade and sampling the whole area using the appropriate density. Then, for each generated point, the algorithm checks that the distance from the track falls into the cascade being processed. Further, it's checked that the point hit the correct surface (splat map), so that we do not spawn trees on a field etc. There is also a vegetation mask that allows for a precise control of where inside a biome the vegetation is allowed to spawn, so that we can cull it around the track.

The vegetation mask is essentially a list of triangles from the meshes that create the track and sidewalks where we prohibit the creation of vegetation. To speed the process up, we precompute a 2D grid of points, where each point holds a list of all the triangles where vegetation should not be spawned. Then, when masking a point, we simply calculate the nearest point of the grid and check all of its associated triangles. The result is demonstrated in Figure 7.

As we spawn trees denser closer to the track, we now have a slight visual issue, because when viewing a forest from its side, which can easily happen when riding along the track, it is very noticeable that the farther parts of it are way sparser. Therefore, we implement one more trick to make the edges



Fig. 8. Distance cascades with forest depth. Most of the vegetation is near the edges of the forest and close to the road.

of the forest denser. We compute a depth map of the forest, which for each point of the forest holds the distance from the nearest edge of the forest (so that the middle of the forest has the greatest values). This is done similarly to the distance function computation shown in Figure 5.

Afterwards, we select a constant distance (5 to 10 meters), and for all of the points of the forest, where its depth is smaller than this constant, we divide its distance from the track by another constant of our choice (approx. 5). This makes it so that the edges of the forest are now perceived as being closer to the track than they really are by our cascading algorithm, therefore they are populated by trees more densely. As a sidenote, this appearance mimics how real world forest grow, as the edges of those are also usually denser, because more light reaches the trees. The result of this implementation trick can be seen in Figure 8.

Finally, we spawn the grass. The overall approach is similar as with trees, with the important difference that significantly more grass patches are present on a map. This makes even the generation of the positions a potential bottleneck. To solve this, we compute a single tile of grass positions (100×100 meters) and repeat it throughout the whole map. This cannot visually be distinguished from a proper position generation. A performance challenge remains associated with the grass spawning itself, and is dealt with in the next section.

To increase visual fidelity, the generator also supports the generation of various flowers, which are meant to make the flora less monotonous. To efficiently render multiple different types of flowers as a single entity (from the point of view of the GPU), we utilize a texture array (e.g., 4×4 , containing 16 distinct textures), which allows us to efficiently swap these texture on the fly by simple indexing. Each flower instance requires an appropriate texture index, which is determined using the 3D position as a seed. To ensure all flowers in a given frame share the same texture, we adjust the flowers' positions to achieve the desired index.



Fig. 9. Demonstration of our custom grass shader with various types of flowers.

Lastly, a rendering optimization is applied. As highly prominent flowers remain visible at greater distances, while less noticeable ones do not, we cull the less prominent flowers earlier to reduce the amount of spawned assets without losing visual fidelity. To enhance flower clustering, we use a simple Perlin noise and only place flowers in areas where the noise value reaches a certain threshold. See Figure 9 for an example.

F. Superbake – pre-runtime optimizations

Our method produces vast amounts of repeating objects, not far in orders of magnitude from those of open-world games. If we used the standard Unity GameObject workflow, the generation process would not be performant enough, resulting in long loading times and big scene files. Most of our instances are grass patches, which do not have any other functionality except for being rendered (no colliders, no gameplay usage). Further, distant objects are unlikely to be interacted with.¹

Hence, we can overcome all the mentioned issues by not instancing distant objects as GameObjects, but only computing and storing their transform matrices and instancing them on the GPU. We scan the scene and detect repeating distant objects. For these, it is sufficient to store their transform matrix, so that their associated `Renderers`, as well as unnecessary scripts, can be removed. We can also remove their `Colliders`, as we can be sure that these objects will never be interacted with. Figure 10 visualizes the `Colliders` that remain in a scene, comparing it with the same scene during runtime.

Often, hundreds of thousands of objects need to be animated (e.g., grass patches, flags, and fans). To handle this, we use noise functions in shaders (for grass and flags), or animations baked into a texture (for fans). Variety can be increased by using texture arrays (at almost no additional performance cost) with the world position as a seed for the array index.

Finally, the size of the saved scene is reduced by saving only the indispensable information about asset transformations and calculating the final matrices at runtime. For example, trees only need to have their x and z positions stored, as the y (height) information can be computed from terrain height.

¹The player is returned to the track after several seconds of off-road driving, which limits the maximum achievable distance from the track.



Fig. 10. Top: Visualization of `Colliders` after the Superbake step. The `Colliders` and the track are highlighted in green. We can see that `Colliders` for trees, houses, and other assets closer than 50 meters from the track are kept in the scene. Bottom: The same camera angle at runtime.

Rotations can be random, hence they need not be stored, and all scale axis are equal, hence only one value needs to be saved. The efficiency of this reduction can be further improved by putting all the values into a single byte array and compressing the array before storing it.

V. EVALUATION

The performance of our implementation was measured on a consumer grade laptop² at a 1920×1080 resolution. Let us now discuss the goals that we set out in Section I-A.

1) *Controllable generation*: The system allows full control over the shape and width of the track at any point of the terrain. The generator features six biomes (e.g., city, forest, meadows) that are automatically spawned on the map and can be changed in any way needed. Further, most of the assets spawned are implemented in various alternatives (e.g., fences, sidewalks, safety features) and the system allows for a simple introduction of new assets. Examples are shown in Figure 11.

2) *Generator performance*: We used 13 fully generated maps for the measurements. Their average size was 0.87 km^2 (range: $0.26\text{--}3.92 \text{ km}^2$), and the average length of a race track was roughly 2.4 km (range: $0.7\text{--}6.9 \text{ km}$).

²i5-13500H 2.60 GHz CPU, 16 GB RAM, Nvidia 4060 (Mobile)



Fig. 11. Examples of possible generator configurations. Multiple different tracks are shown in this compilation, showcasing different asset variants.

Two different metrics were used for the evaluation. The first metric consisted of measuring the time required to generate 1 square kilometer of a track and its surroundings. This metric can be slightly misleading since further surroundings (farther from the track) are much easier to generate, and it is up to discussion whether or not they should be considered. In our evaluation, we opted to measure the area of the track as the track's bounding box extended by 150 meters on each side (as that is the distance where most of the generation takes place). The time measured ranged from 45 to 155 seconds per square kilometer, averaging at 90 seconds.

The second metric is time per kilometer of the track itself. As most of the generation takes place close to the track, this metric might provide a better idea about the generator performance and how it depends on the playable area. The time measured ranged from 12 to 52 seconds per kilometer, averaging at 29 seconds. Finally, the average time per the generation of a whole map averaged at 67.2 seconds. Based on this data, we believe that the system is performant enough to allow for an interactive level design.

3) *Runtime performance*: All runtime performance measurements were conducted within *Motorbike Evolution 2024*³, a commercially released racing game. Thirteen tracks and environments fully generated by our system are used in the game, demonstrating its viability in a real production setting, both in terms of visual fidelity and runtime efficiency. The measurements were done with the following setup: Unity 6000.0.23f1 LTS, HDRP 17.0.3, three shadow-cascades updating at demand, post-processing with volumetric effects, deferred rendering path, DLSS 3 without frame generation.

During gameplay, frame rates consistently ranged from 236 FPS (4.24 ms per frame) to 280 FPS (3.57 ms per frame), with an average of 262 FPS (3.82 ms per frame). The number of draw calls remained consistently under 200, averaging around 160 and reaching as low as 130. All 13 tracks load and initialize in under 2 seconds. Disk space usage was measured at 8.93 MB per kilometer of race track and 27 MB per square kilometer.

VI. CONCLUSION

We have presented a controllable race track surroundings generator that significantly accelerates the level design process while maintaining high visual fidelity and designer control. Our system generates densely populated, high-quality track

surroundings in a matter of minutes. The generation process is quick enough to enable iterative race track design and the resulting map is well optimized for runtime usage in real time video games. The novelty that this work brings is giving full control over a track's layout to the designer, and adapting the generated content based on that. The tool not only generates the surrounding terrain and vegetation, but also spawns assets in a context-aware manner.

In the future, we would like to evaluate the usability of the generator with level designers, and to conduct a user study comparing our work with commercially available generators.

REFERENCES

- [1] MasterPixel3D, "Race Track Generator", Unity Asset Store, 2018.
- [2] I. Hudson, "Race Track Generator", GitHub, 2019.
- [3] G. Maciel, "Procedural Racetrack Generation", Unity Asset Store, 2020.
- [4] D. Loiacono, L. Cardamone, and P. L. Lanzi, "Automatic Track Generation for High-End Racing Games Using Evolutionary Computation", *IEEE Transactions on Comp. Intelligence and AI in Games*, 2011.
- [5] U. Göhner and F. Behrens "Procedural race track generation for domain randomization", 2020.
- [6] Unity Technologies, "Unity 6000.0.23f1", Game development platform, 2024.
- [7] A. Cristea and F. Liarokapis, "Fractal Nature - Generating Realistic Terrains for Games", 7th International Conference on Games and Virtual Worlds for Serious Applications (VS-Games), 2015, pp. 1-8.
- [8] R. Fischer, J. Boeckers, and G. Zachmann, "Procedural Generation of Landscapes with Water Bodies Using Artificial Drainage Basins", *Advances in Computer Graphics. CGI 2022. Lecture Notes in Computer Science*, vol 13443. Springer, Cham.
- [9] R. Fischer, P. Dittmann, R. Weller, et al., "AutoBiomes: procedural generation of multi-biome landscapes", *Vis Comput* 36, 2020.
- [10] F. Aurenhammer, "Voronoi Diagrams—a Survey of a Fundamental Geometric Data Structure", *Association for Computing Machinery*, 1991.
- [11] L. Cardamone, P.L. Lanzi, and D.Loiacono, "TrackGen: An interactive track generator for TORCS and Speed-Dreams", *Applied Soft Computing*, 28, 550-558, 2015.
- [12] Awesome Technologies, "Vegetation Studio Pro", Unity asset, 2018.
- [13] C. Newlands and K.-P. Zauner, "Procedural Generation and Rendering of Realistic, Navigable Forest Environments: An Open-Source Tool", 2022.
- [14] F. Freiknecht and W. Effelsberg, "A Survey on the Procedural Generation of Virtual Worlds", *Multimodal Technologies and Interaction*, 2017.
- [15] F. Freiknecht, "Terrain Tutorial Using Shade-C", 2011.
- [16] K. Perlin, "An Image Synthesizer", *Computer Graphics*, 1985.
- [17] F. Queißner and M. Kochkarov, "TUST Scripting Library", 2013.
- [18] C. Sarndal, B. Swensson, and J. Wretman, "Model Assisted Survey Sampling", Springer-Verlag, 1992.
- [19] P. Casteljau, "de Casteljau's algorithm", *Computer Aided Geometric Design*, 1959.
- [20] C. d. Boor, "A Practical Guide to Splines. Springer-Verlag", Springer-Verlag, 1978.
- [21] D. P. Kroese, T. Brereton, T. Taimre and Z. Botev, "Why the Monte Carlo method is so important today", *Computer Science, Education*, 2014.
- [22] W. J. Morokoff and R. E. Caflisch, "Quasi-Monte Carlo integration", *Mathematics Department, University of California*, 1995.

³P. Šimůnek, Video game, Self-published, 2024